

Protokoll UE2 Treecheck

1. Baum einlesen

1. Inersion:

- aus den Folien der Vorlesung übernommen
- best case: $O(1)$ (wenn direkt bei Root eingefügt)
- average case: $O(\log n)$ → Baum ist gleichmäßig verteilt, ich muss nicht JEDEN Knoten zum einfügen durchlaufen
- worst case: $O(n)$ → Baum ist eine einfach verkettete Liste
- Beschreibung: geht von der Wurzel aus die Knoten entlang, entweder nach links oder rechts, je nachdem ob der Wert größer oder kleiner ist, als der aktuelle Knoten

2. heightAndBalance:

- best/average/worst case: $O(n)$ → jeder Knoten wird einmal besucht
- Beschreibung: Post Order → rechnet vom letzten Knoten zum darüber liegenden dazu; vergleicht, welcher Wert größer ist und gibt diesen an den darüber liegend Knoten weiter, usw.

3. findMin/findMax:

- best/average/worst case: $O(n)$ → jeder Knoten wird einmal besucht
- Beschreibung: Post Order → liefert dem Knoten von unten die Werte von seinem left/right vergleicht diese und nimmt den größeren für max, kleineren für min. Dieser Wert wird wieder an den darüber liegenden Knoten gegeben zum Vergleichen, ect.

4. findSum:

- best/average/worst case: $O(n)$ → jeder Knoten wird einmal besucht
- Beschreibung: Post Order wie bei den vorherigen Funktionen; Rechnen aktuellen Wert zusammen mit den Werten von links und rechts und gibt es nach oben weiter.

5. countEntries:

- best/average/worst case: $O(n)$ → jeder Knoten wird einmal besucht
- Beschreibung: Wieder Post Order; rechnet einfach immer eins zum Knoten dazu und gibt den Wert zum Knoten darüber weiter.

2. Suche im Baum:

1. searchKey:

- best case: $O(1)$ → gesuchter Wert ist direkt in der Wurzel
- average case: $O(\log n)$ → Baum ist gleichmäßig verteilt, es wird nur ein Pfad durchsucht (nicht der ganze Baum)
- worst case: $O(n)$ → Baum ist eine verkettete Liste, jeder Knoten muss überprüft werden
- Beschreibung: Die Funktion dient als Schnittstelle zur Suche und ruft die rekursive Hilfsfunktion searchKeyHelper auf.
Abhängig vom Ergebnis wird ausgegeben, ob der gesuchte Schlüssel im Baum enthalten ist. Im Erfolgsfall wird zusätzlich der Pfad vom Wurzelknoten bis zum gefundenen Knoten ausgegeben.

2. searchKeyHelper:

- best case: $O(1)$ → gesuchter Wert ist direkt in der Wurzel
- average case: $O(\log n)$ → Baum ist gleichmäßig verteilt, es wird nur ein Pfad durchsucht (nicht der ganze Baum)
- worst case: $O(n)$ → Baum ist eine verkettete Liste, jeder Knoten muss überprüft werden
- Beschreibung: Durchsucht den Baum rekursiv nach einem Schlüssel und speichert dabei den Pfad. Gibt die Länge des Pfades zurück oder 0, falls der Schlüssel nicht gefunden wurde.

3. searchSubtree:

- best case: $O(m)$ → Subtree wird direkt an der Wurzel gefunden, nur dieser Teilbaum wird verglichen
- average case: zwischen $O(m)$ und $O(n \cdot m)$ → abhängig davon, wie früh der Subtree gefunden wird
- worst case: $O(n \cdot m)$ → für jeden Knoten im großen Baum wird der Subtree vollständig verglichen
- Beschreibung: Die Funktion durchsucht den gesamten Baum rekursiv und prüft an jedem Knoten, ob dort der gegebene Subtree beginnt. Dazu wird die Funktion isSameTree verwendet, um die Teilbäume zu vergleichen.

4. isSameTree:

- best/average/worst case: $O(m)$ → jeder Knoten des Subtrees wird einmal besucht und verglichen
- Beschreibung: Vergleicht zwei Teilbäume rekursiv und prüft, ob sie in Struktur und Werten vollständig identisch sind.